Project Proposal

Title: Dynamic Batch-Size Throttling for GPU di/dt Noise Mitigation

Students: Nikhil Reddy Mummadi, Jakub Mroczek

---

1. Introduction

Modern AI workloads, particularly GEMMs used in LLMs and convolutional networks, induce massive, near-instantaneous current demands on GPU Power Delivery Networks (PDNs). These fast load transients di/dt cause severe voltage droops. If the voltage drops below the critical threshold Vmin, the GPU may experience timing failures or crash. Traditional mitigation relies on pessimistic voltage guard bands or conservative hardware throttling, both of which leave significant performance on the table.

This project proposes a software-driven loop control system that actively mitigates di/dt noise. By building a digital twin of the GPU Power Delivery Network (PDN) and wrapping it in a closed-loop controller, we aim to dynamically throttle the batch size of streaming inference workloads in real-time. This ensures the workload dynamically scales back to prevent Vmin violations during high-power transients and scales up to maximize throughput during stable periods.

2. Methodology

The project involves the following components:

- **GEMM Workloads (C++/CUDA):** We will develop 6-8 custom GEMM kernels (ranging from naive to highly optimized cuBLAS) compiled into a shared object library. To simulate realistic streaming inference without processing duplicate data, the kernels will be utilizing a sliding-window memory pointer updated by the host.

- **GPU PDN (Python):** A continuous loop will poll the physical GPU power draw via the NVIDIA Management Library (pynvml) while the GEMM kernels run. The sampled physical power will be passed through a RLC circuit simulation (using scipy.signal) representing the package and board PDN. This generates the simulated voltage droop (Vdroop).

- **The Closed-Loop Controller (Python):** A threshold-based controller will evaluate the simulated voltage. If Vdroop approaches Vmin, the controller will instantly throttle the subsequent kernel launch's batch size. If the voltage recovers, the controller will incrementally increase the batch size to maximize streaming throughput. The PDN and the closed loop controller shall be implemented in Python for rapid prototyping and visualization.

- **Controller Extension (C++) (subject to timeline):** A controller written in native C++ will be used for performance analysis between interpreted and compiled control loops.

Python will be the master host to dynamically throttle batch sizes and launch C++/CUDA kernels using Python's ctypes library as the bridge with C++. There is potential for race conditions with the outlined methodology, outlined below are the risks and possible features to mitigate them:

Risk	Methodology improvment
CPU queues kernel faster than GPU can process, wrong batch size gets queued.	Synchronization barrier between kernel launch and batch size calculation. Possibly only enforced at high power.
Short kernel time could make it difficult to pinpoint which batch size correlates to power reading. Could lead to pessimistic batch size recommendations or jitter.	Adding hysteresis to the system can smooth feedback loop.
di/di events can happen too quickly for CPU to poll, can lead to lower power readings.	May need determine an additional guard band.

3. Objectives and Timelines

The project will be executed four phases:

- **Weeks 1-2 (Early March): Workload and Python-C++ Bridge Initialization.**
  - Develop baseline C++/CUDA GEMM kernels, and tune for maximum di/dt transient.
  - Establish the ctypes bridge to launch kernels from the Python host.
  - Implement the streaming data input buffer.
- **Weeks 3-4 (Mid-March): Kernel Profiling.**
  - Sweep batch sizes from across different GEMM kernels.
  - Log physical NVML power draw to map the Power vs Batch Size.
- **Weeks 5-6 (Early April): PDN Modeling.**
  - Implement the RLC package model using scipy.
  - Feed kernel power data into the model to simulate realistic Vmin droop events.
- **Weeks 7-8 (Late April): Integrate Control Loop with PDN during kernel runs.**
  - Integrate the control logic to dynamically update the batch size based on simulated voltage.
  - Introduce a FIFO buffer to model realistic PDN RC delay.
  - Generate stacked timeline visualizations (Power, Batch Size, Voltage).

4. Success Rubrics

The success of this project will be measured against the following engineering criteria:

1. **Workload Launch:** Successful execution of C++ CUDA kernels from a Python host using ctypes bridge without memory segmentation faults.
2. **Batch Size Sensitivity:** Prove that highly optimized GEMM kernels generate di/dt transients at high batch sizes.
3. **Di/dt Mitigation:** The closed-loop controller must show success in preventing simulated voltage drops below Vmin by throttling batch size under peak workload transients.
4. **Throughput Recovery:** The controller must also recover and maximize batch size once the transient noise settles, without generating more transient noise during load step ups.

5. Resources Required

- **Hardware:** NVIDIA GPU with NVML telemetry support (from Purdue Systems)
- **Software Stack:**
    - CUDA Toolkit (for compiling .cu files).
    - Python 3.x with the following libraries: pynvml (telemetry), ctypes (C++ bridging), scipy (RLC differential equations), and matplotlib (data visualization).

6. References

- Liang, Zhirui, et al. “GPU-To-Grid: Voltage Regulation via GPU Utilization Control.” ArXiv.org, 2026, arxiv.org/abs/2602.05116.